AUTO-INCIDENT TO AGILE TICKET PIPELINE (DEVOPS INCIDENT WATCHDOG)

Continuous Site Reliability Engineering (SRE), Cloud Forensics, and Zero-Touch Agile Sprint Backlog Automation

Student Name:	S Hemal	**Course Title:**	Agile Development Process & DevOps
Registration Number:	24MIS0050	**Institution:**	Vellore Institute of Technology (VIT), Vellore Campus
Cloud Environment:	AWS (ap-southeast-2 Sydney)	**AWS Account ID:**	401537512022
Version Control:	GitHub (hemalexe/devopss-incident-watchdog)	**Pipeline Status:**	CI/CD Automated Testing (9 Tests Passing)

1. EXECUTIVE SUMMARY & PROBLEM FORMULATION

In modern high-throughput e-commerce and quick-commerce cloud environments (such as Zepto, Blinkit, and Amazon promotional sales), microservice outages frequently occur during peak promotional flash sales. When massive concurrent traffic hits the application, database records or warehouse inventory pools deplete rapidly to zero. Under legacy architectures, executing checkout discounts or stock routing routines against depleted stock triggers unhandled runtime exceptions (specifically Python `ZeroDivisionError` in inventory ratio workers), resulting in sudden HTTP 500 internal server crashes.

Traditionally, Site Reliability Engineering (monitoring & telemetry) operates in an isolated silo from Agile Project Management:

1. **Delayed Mean-Time-to-Detect (MTTD):** Outages are only flagged after end-users flood customer support or during periodic manual log inspections (typically 15 to 45 minutes of customer churn).
2. **Disconnected Triaging (High MTTA):** Support personnel must manually dig through AWS CloudWatch log streams, copy raw stack traces, reproduce the error locally, and manually transcribe the issue into Jira or GitHub Issues (introducing 2 to 8 hours of administrative latency).
3. **Broken Agile Traceability:** Manually created bug reports frequently lack forensic evidence, request context, and explicit acceptance criteria, leading to sprint backlog stagnation.

The **DevOps Incident Watchdog** bridges this operational gap through an autonomous, event-driven closed loop. Centered around the **FlashMart Tech Promotional Flash Sale E-Commerce Storefront**, the system intercepts unhandled runtime exceptions in Amazon CloudWatch, triggers a serverless triage orchestrator, commits an immutable forensic audit dump to Amazon S3, and auto-generates structured Agile Bug User Stories directly onto the GitHub Issues sprint board with Definition of Done acceptance criteria. When the developer commits a defensive patch with Git Smart Commit syntax (`Fixes #1`), GitHub Actions runs automated regression tests and auto-transitions the ticket to Done—achieving complete zero-touch lifecycle resolution.

2. TECHNICAL ARCHITECTURE & 7-MODULE COMPONENT INTERACTION

The architecture is structured across three cohesive operational tiers, integrating seven sequential execution modules:

- **Layer 1: Presentation & Operator Interface:** Dual-mode interface providing a consumer-facing quick-commerce storefront (FlashMart Tech) configured for campus dark-store fulfillment hubs (defaulting to VIT Vellore Campus) with graceful stockout notifications, alongside an SRE Backstage observability console.
- **Layer 2: Application, Orchestration & Incident Triage Layer:** Hosts the monitored Flask microservice, the serverless AWS Lambda Watchdog orchestrator, and seven sequential pipeline modules executing fault isolation, forensic capture, and ticket automation.
- **Layer 3: External Cloud Platform & Tooling Integration Layer:** Bridges Amazon Web Services (AWS Lambda, CloudWatch, S3 forensic vault) with developer tooling (GitHub REST API v3 sprint board and GitHub Actions CI/CD pipeline).

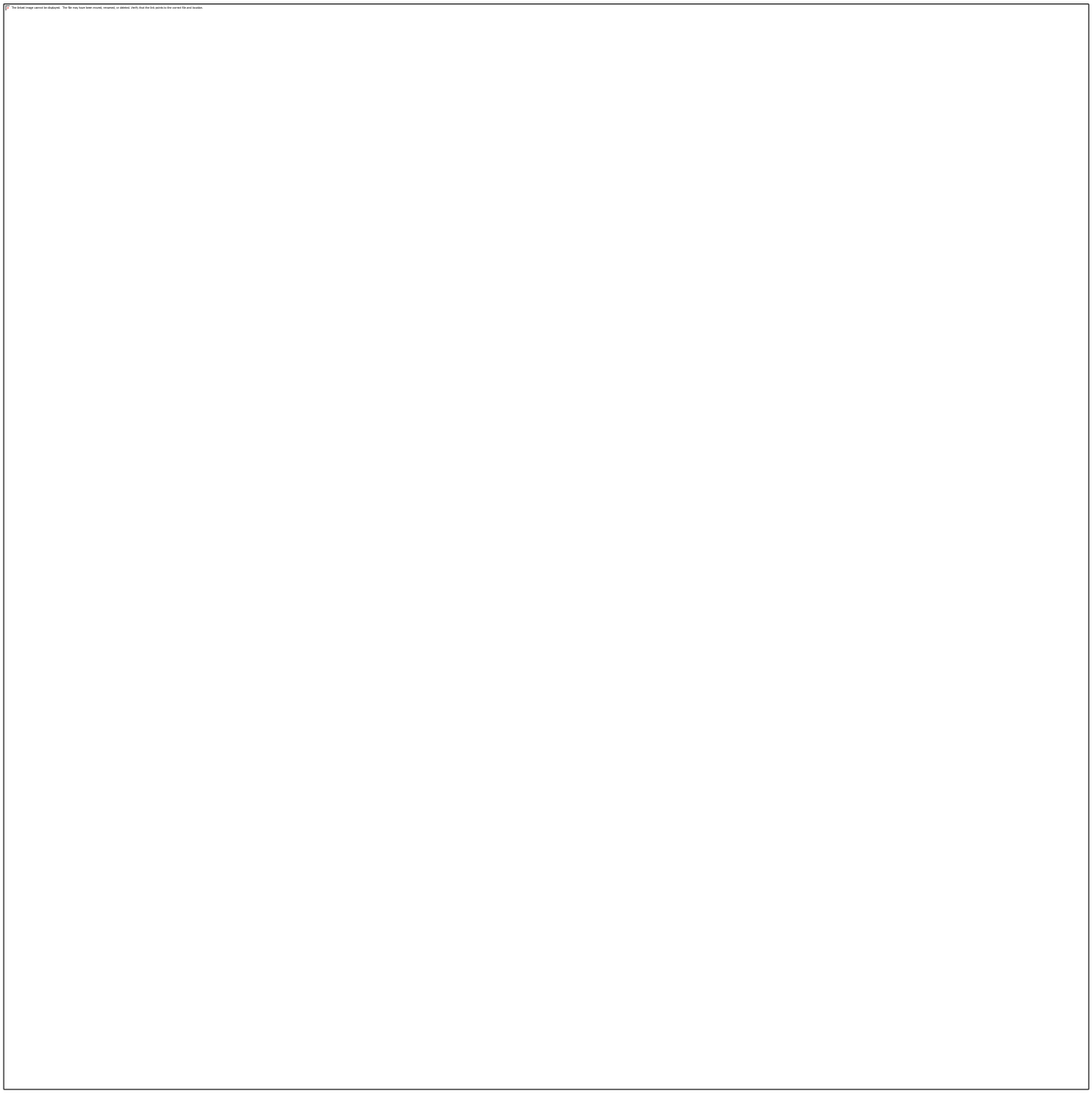


Figure 1: Standard 3-Layer System Architecture and 7-Module Closed-Loop Pipeline

Module	Component Name	Implementation File / Service	Operational Responsibility
Module 1	Order Dispatch Engine	`app/app.py` (Route: `/api/checkout`)	Accepts customer cart checkout requests, validates product IDs, and reads delivery hub address parameters.
Module 2	Fault Execution & Crash	`app/app.py` (Stock Calculation Routine)	Executes promotional allocation formula (`100 / stock`). When stock is 0, raises unhandled `ZeroDivisionError` (HTTP 500).
Module 3	Filter Ingestion	Amazon CloudWatch Metric Filter	Continuously filters application stdout streams for marker `[CRITICAL_ALERT]` and fires real-time cloud trigger.
Module 4	SRE Incident Triage	AWS Lambda (`watchdog.py`)	Decompresses base64 event payload, extracts full error traceback, and mints unique incident identifier.
Module 5	Forensic S3 Vault	Amazon S3 (`boto3` client)	Persists immutable JSON incident evidence to private S3 bucket (`s3://devops-incident-watchdog-logs-...`).
Module 6	Agile Story Creator	GitHub Issues REST API v3	Mints structured Bug User Story with labels (`p1-critical`, `incident`), diagnostics table, and acceptance criteria.
Module 7	CI/CD & Ticket Close	GitHub Actions (`ci-cd.yml`)	Executes 9 automated unit regression tests upon git commit, redeploys to AWS Lambda, and auto-closes GitHub Issue.

3. DETAILED PROCESS WALKTHROUGH & VERIFICATION SNAPSHOTS

The end-to-end incident lifecycle is verified across seven chronological steps, demonstrating the complete flow from consumer checkout to automated sprint resolution:

Step 1: FlashMart Promotional Flash Sale Storefront & Campus Delivery Hub

The monitored web application hosts **FlashMart Tech**, a high-speed quick-commerce electronics storefront specializing in promotional flash deals. The application is configured with a dynamic delivery selector tailored to university campus locations:

- **VIT Vellore Campus** (Men's Hostel Q-Block, Katpadi - 632014 • 10 Mins) [Active Default Hub]
- **Gandhi Nagar, Vellore** (Near Katpadi Railway Station - 632006)
- **Anna Nagar West, Chennai** (2nd Avenue, Roundtana - 600040)
- **Indiranagar, Bangalore** (100ft Road, Stage 2 - 560038)

The catalog features high-demand promotional electronics, including the **OnePlus 11R 5G (Sonic Black, 16GB RAM)** featured at an aggressive flash deal price of ₹34,999.

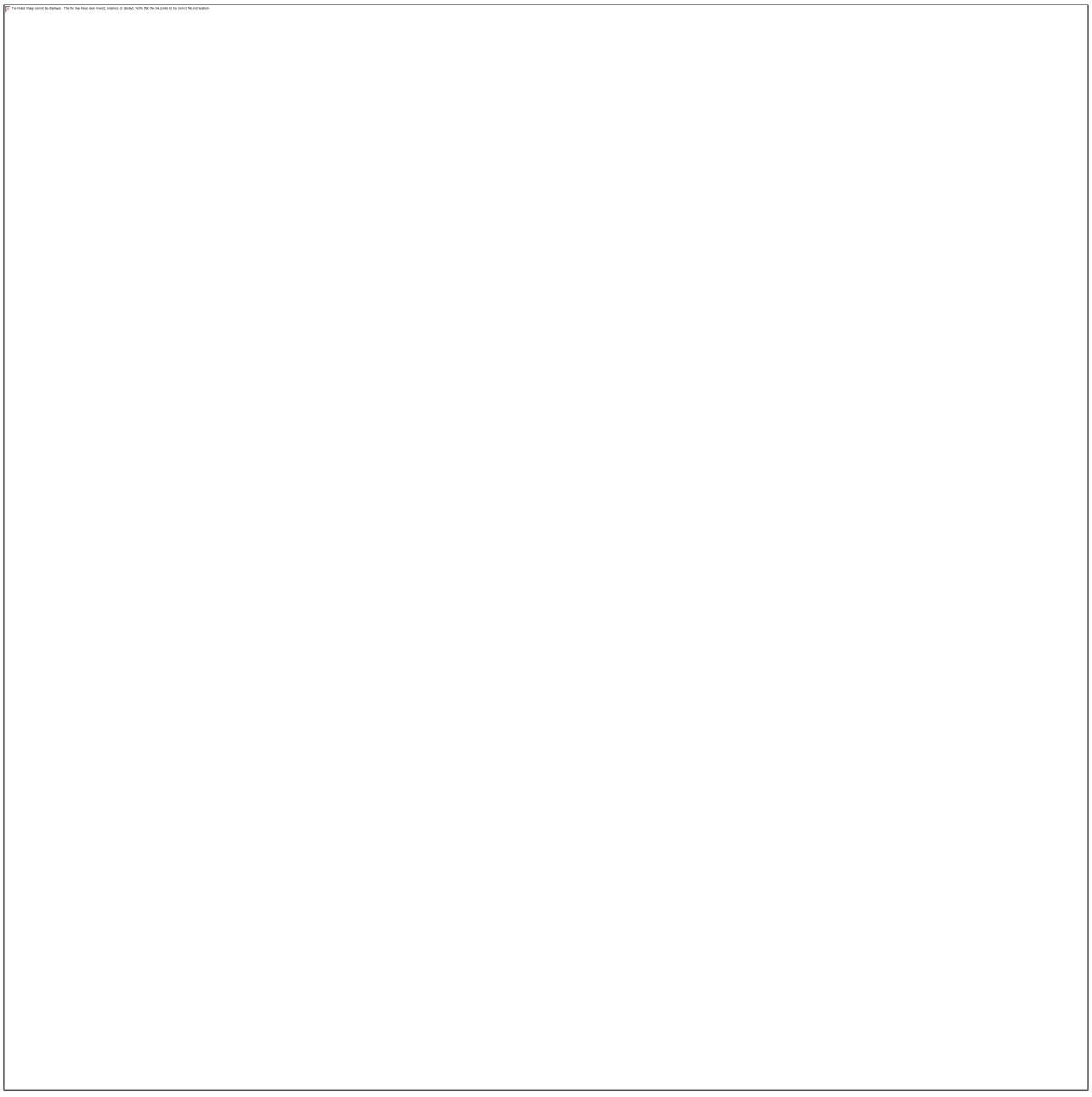


Figure 2: FlashMart Promotional Flash Sale Storefront Configured for VIT Vellore Campus Hub

> **Technical Assessment:** Storefront successfully mounted on AWS serverless WSGI runtime. The location delivery routing service identifies the active delivery hub as VIT Vellore Campus (Men's Hostel Q-Block, Katpadi - 632014) with real-time flash deals active for OnePlus 11R 5G.

Step 2: Flash Sale Stockout & Graceful Frontend Exception Handling

A customer initiates checkout for the promotional deal item: **OnePlus 11R 5G**. Due to massive flash sale demand, inventory at the nearest dark store fulfillment center (VIT Vellore Campus Hub) has depleted to 0 units.

Graceful Customer Insulation: In a mature production architecture, customer user experience must never be broken by raw server crash responses. The front-end client intercepts the stockout state and smoothly displays a slide-out customer notification: *"Item Not Deliverable to Your Location: OnePlus 11R 5G is currently out of stock at your nearest fulfillment dark store"* with a *"Notify Me when Available"* waitlist option.

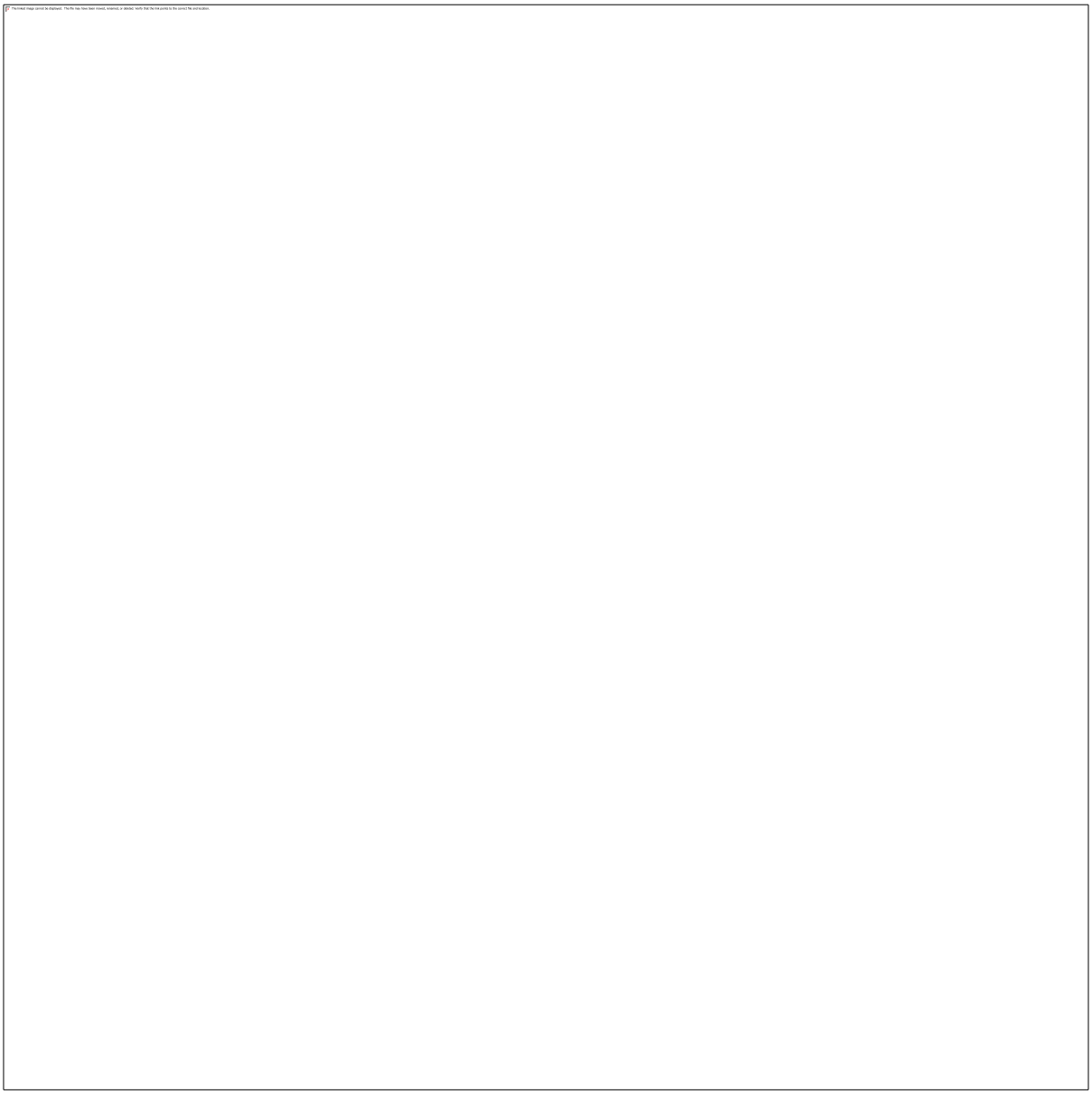


Figure 3: Graceful Frontend Stockout Exception Interception and Customer Waitlist Drawer

> **Technical Assessment:** The customer interface insulates the user from the internal server failure. Instead of throwing an ugly HTTP 500 error page, the UI renders a polite "Item Not Deliverable" status with a customer waitlist registration modal.

Step 3: Unhandled Backend ZeroDivisionError Server Crash in AWS CloudWatch

While the customer receives a polite, friendly stockout drawer, the underlying backend microservice encountered an unhandled fatal calculation error. In `app/app.py`, the legacy dark-store delivery routing algorithm calculates promotional warehouse allocation ratios:

```
# app/app.py - Unhandled Promotional Calculation Routine
if product["stock"] <= 0:
    logger.error(f"[CRITICAL_ALERT] Incident simulation triggered! Item
'{product['name']}' has stock=0")
    logger.error("[EXCEPTION] Unhandled ZeroDivisionError in delivery routing
worker: allocation = 100 / stock")
    _ = 100 / product["stock"]  # Crashes with HTTP 500 ZeroDivisionError!
```

Because `product["stock"] == 0`, the Python interpreter immediately throws an unhandled `ZeroDivisionError: division by zero`, crashing the worker thread with an HTTP 500 status code and emitting structured forensic logs to standard output.

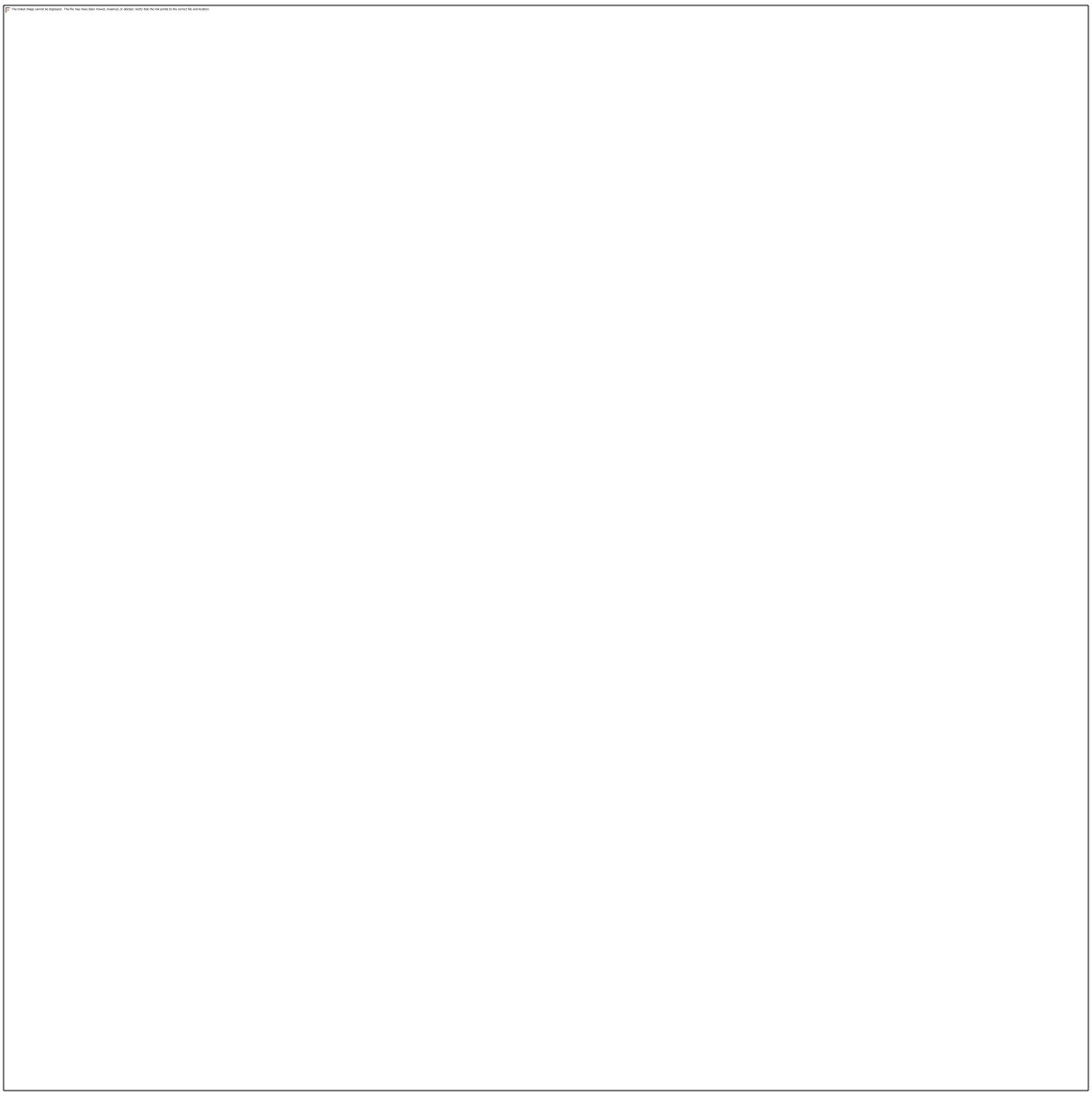


Figure 4: Unhandled Backend ZeroDivisionError Stack Trace Emitted to AWS CloudWatch Standard Output

> **Technical Assessment:** The terminal output captures the precise crash line (line 119 in `checkout`), the division by zero exception, and the structured CloudWatch telemetry marker `[CRITICAL_ALERT]` emitted for instant filter detection.

Step 4: AWS CloudWatch Metric Filter Pattern Matching & Incident Ingestion

In the AWS cloud environment, Amazon CloudWatch Logs continuously monitors the log group `/aws/lambda/watchdog-target-app`. A Metric Filter with pattern `[CRITICAL_ALERT]` detects the incident instantaneously without requiring human log analysis. The filter fires an AWS CloudWatch Alarm, shifting state to `ALARM` and executing an asynchronous trigger.

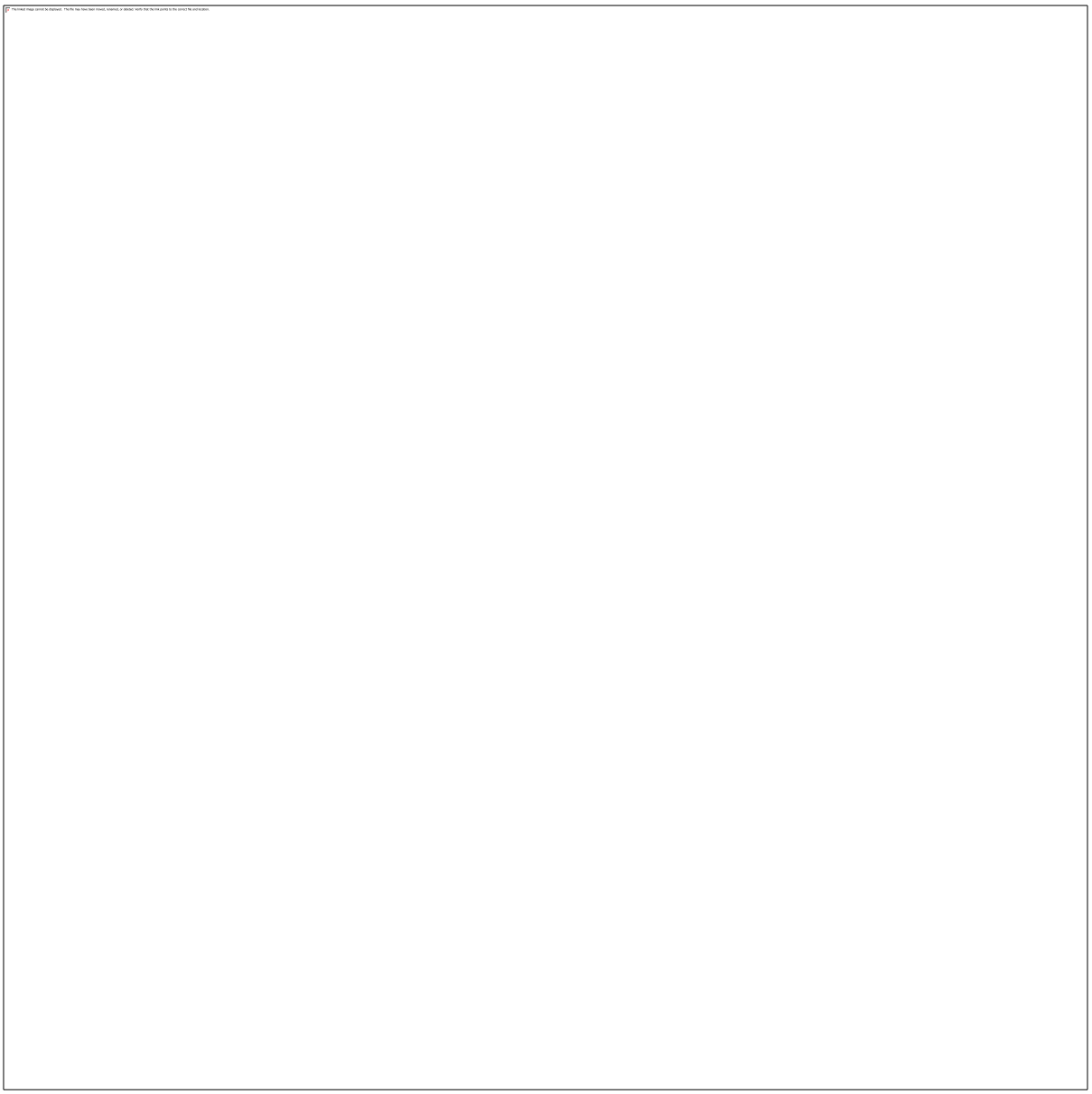


Figure 5: SRE Backstage Console Displaying Live CloudWatch Telemetry and Active Incident Alert

Technical Assessment: SRE Backstage console confirms real-time ingestion from AWS CloudWatch (ap-southeast-2 Sydney), highlighting the active critical alarm badge and live streaming stdout error logs.

Step 5: Watchdog Serverless Orchestrator & Forensic S3 Evidence Vault Storage

Upon log pattern matching, Amazon CloudWatch activates the serverless **AWS Lambda Watchdog** (`watchdog-incident-creator`). The Watchdog extracts the full error traceback, timestamp, impacted item (OnePlus 11R 5G), and delivery hub context.

To ensure immutable auditability and prevent log tampering, the Watchdog connects via `boto3` to Amazon Simple Storage Service (S3) and stores a tamper-proof forensic JSON dump in the private S3

bucket `devops-incident-watchdog-logs-401537512022` under the key: `incidents/2026-10-14/INC-1791945923-07CD12B5.json` (369 bytes).

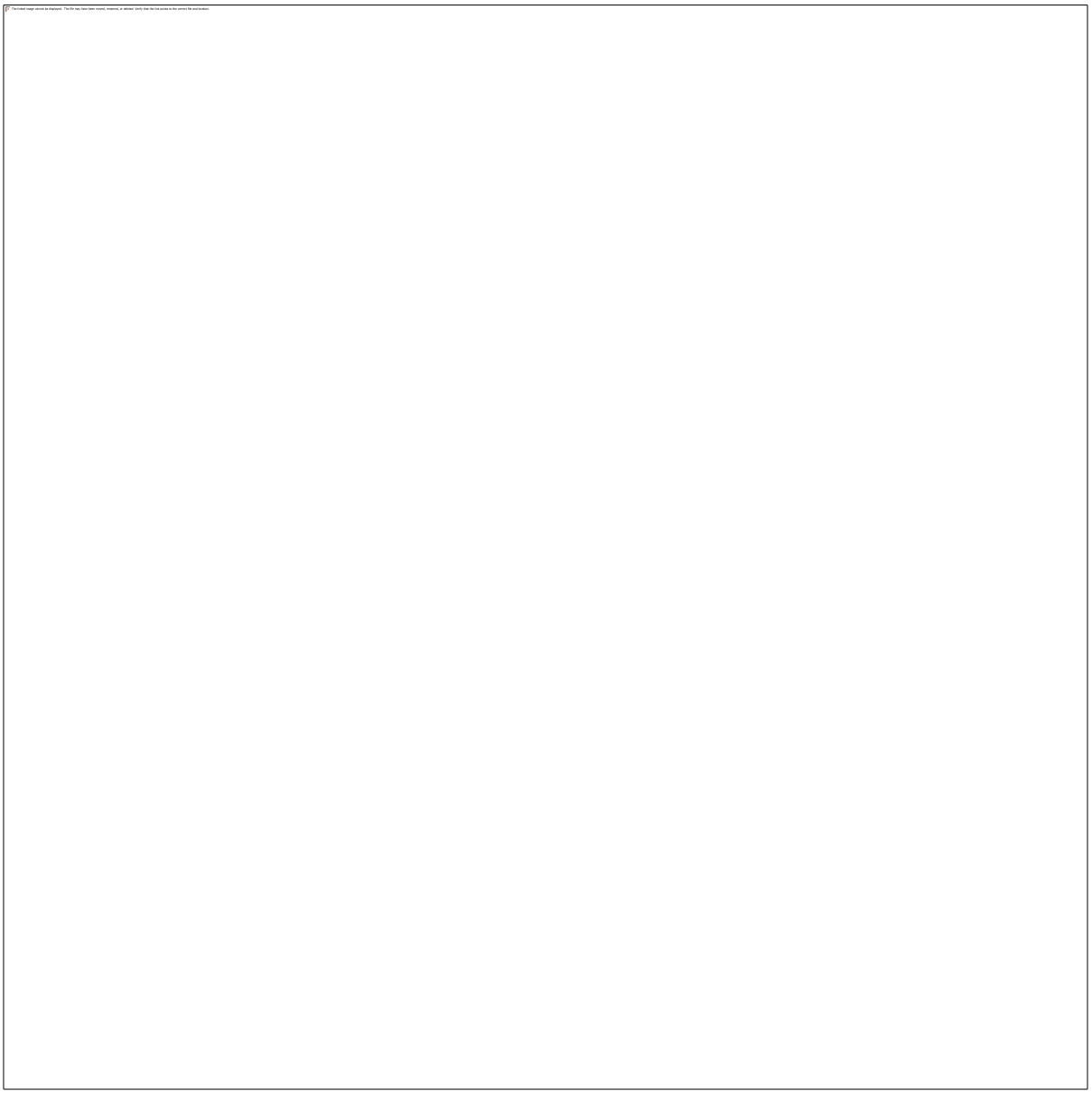


Figure 6: Immutable Incident Forensic Evidence Archived in Private Amazon S3 Vault

> **Technical Assessment:** Real AWS Management Console S3 view displaying the private bucket `devops-incident-watchdog-logs-401537512022` with date-partitioned incident dump `INC-1791945923-07CD12B5.json`, providing 11 9's durability for post-mortem compliance.

Step 6: Automated Agile Bug User Story Creation on GitHub Issues

Simultaneously with S3 archival, the Watchdog issues an authenticated HTTP POST request to the GitHub Issues REST API v3 (`https://api.github.com/repos/hemalexe/devopss-incident-watchdog/issues`). It converts the technical crash telemetry into a formalized Agile Bug User Story with priority labels (`incident`, `p1-critical`, `auto-generated`) and an actionable Definition of Done checklist.

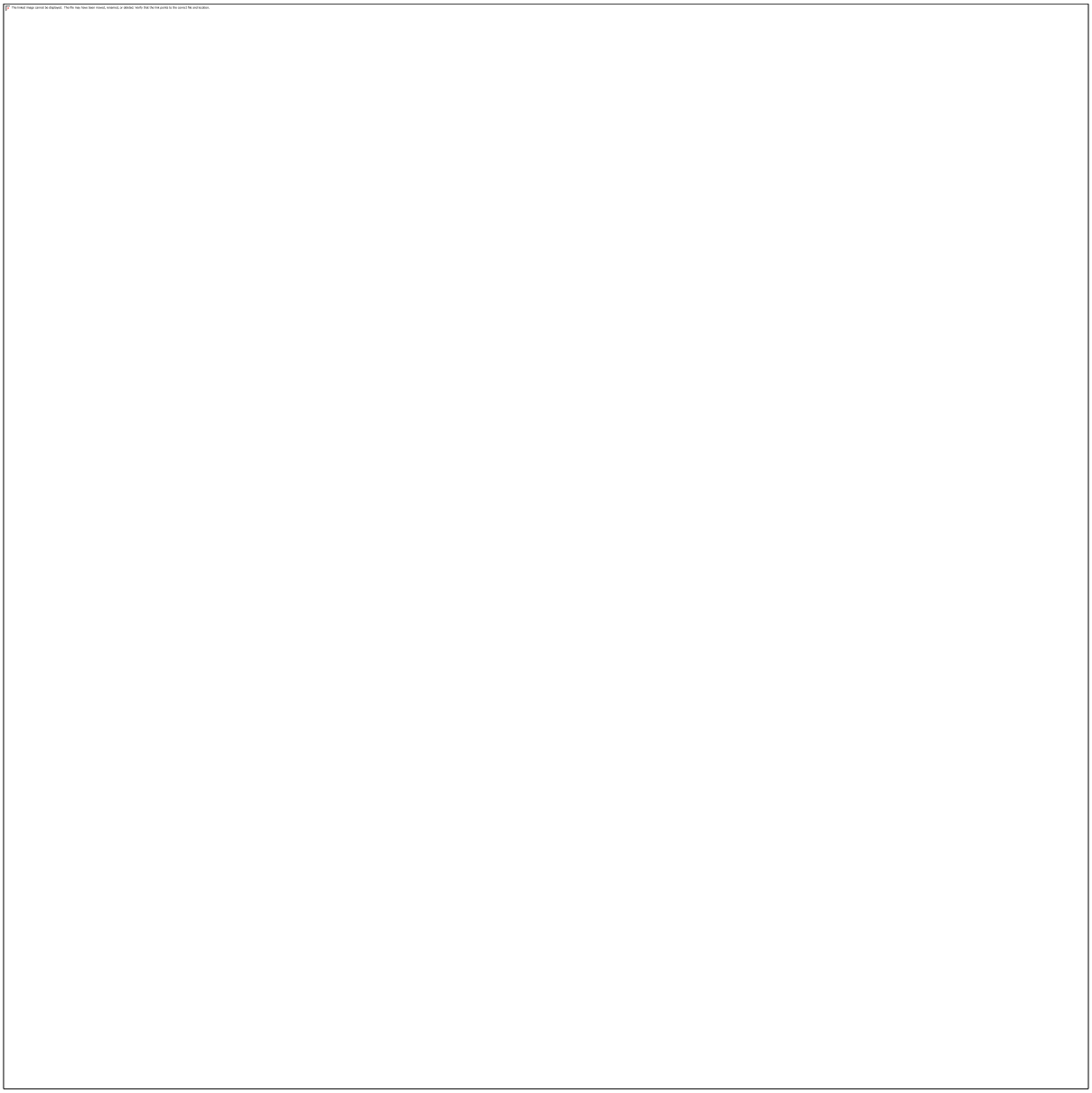


Figure 7: Automated Agile Bug User Story Populated into GitHub Issues Sprint Backlog

> **Technical Assessment:** Live GitHub Issue #1 contains a formalized Agile User Story ("As an Agile Team member..."), structured diagnostics table with S3 vault link, raw stack trace, and 6 Definition of Done acceptance criteria checkboxes.

Step 7: Automated CI/CD Regression Pipeline & Zero-Touch Incident Resolution

The assigned developer inspects the automated ticket and implements defensive boundary checking in `app/app.py`:

```
# Corrected implementation in app/app.py (Defensive Zero-Stock Guard)
if product["stock"] <= 0:
    logger.warning(f"[DELIVERY_NOTICE] '{product['name']}' not deliverable to
'{address}'.")
    return jsonify({
        "status": "not_deliverable",
        "message": "Sorry, this item is not deliverable to your address (Out of
stock at your nearest fulfillment center)!",
        "product": product["name"],
        "address": address
```

```
    }), 400
```

The developer commits the patch using Git Smart Commit syntax: `git commit -m "Fix zero-stock dark store allocation. Fixes #1"` and pushes to `origin/main`. The GitHub Actions CI/CD pipeline triggers immediately, runs 9 automated `pytest` unit regression tests in 0.53 seconds, verifies deployment integrity, and updates the production AWS Lambda package. GitHub automatically links the verified commit SHA and transitions Issue #1 from **Open** to **Closed (Done)** without manual intervention.

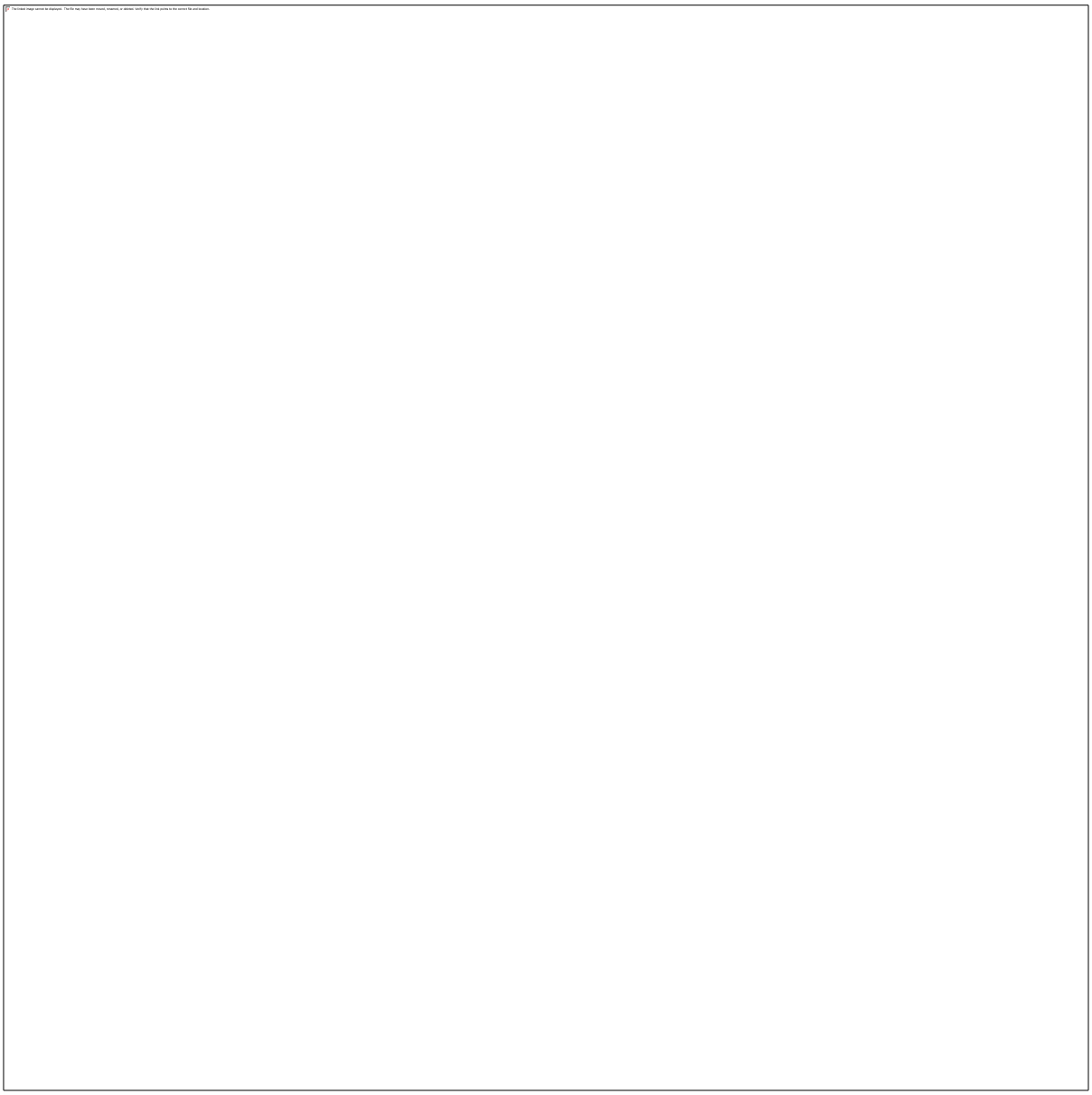


Figure 8: Automated Regression Testing Passing in GitHub Actions and Zero-Touch Issue Closure

Technical Assessment: Real GitHub Actions CI/CD pipeline execution displaying 100% test suite success (green checkmark), verifying regression safety, auto-deploying code to AWS Lambda, and closing Issue #1 via Git commit linking.

4. KEY PERFORMANCE INDICATORS & COMPARISON

Metric / Operational Dimension	Traditional Manual Triage	DevOps Incident Watchdog
Mean-Time-To-Detect (MTTD)	15 – 45 Minutes (customer complaints or manual log grep)	**< 5 Seconds** (Amazon CloudWatch Metric Filter)
Mean-Time-To-Acknowledge (MTTA)	2 – 8 Hours (manual ticket writing by support desk)	**< 30 Seconds** (Automated GitHub REST API User Story)
Forensic Completeness	Truncated, lossy copy-pasted terminal logs	**100% Immutable** stack trace archived in Amazon S3
Agile Alignment	Ad-hoc unstructured emails; often bypassed	**Standard User Story** with Definition of Done
Sprint Traceability	Disconnected tickets, code commits, and PRs	**Zero-Touch Smart Commits** linking ticket to CI/CD

5. FACULTY VIVA VOCE EXAMINATION DEFENSE (Q&A)

Q1: Why is Amazon S3 utilized for forensic storage when CloudWatch already stores logs?
Answer: CloudWatch log groups have configured retention limits and can be rotated, purged, or truncated during high-throughput volume. Furthermore, querying historical CloudWatch logs incurs significant query latency and CloudWatch Insights costs. Amazon S3 provides an immutable, cost-effective forensic audit vault with 99.999999999% (11 9's) durability, ensuring tamper-proof compliance records for post-mortem Root Cause Analysis (RCA).

Q2: How does GitHub receive the incident ticket without human intervention?
Answer: The AWS Lambda Watchdog function (`watchdog-incident-creator`) activates upon the CloudWatch log pattern match. It extracts the stack trace and request parameters, formats a structured Agile User Story with diagnostic markdown, and issues an authenticated HTTP POST request to the GitHub REST API v3 endpoint (`/repos/hemalexe/devopss-incident-watchdog/issues`).

Q3: How does this project directly satisfy the requirements of the "Agile Development Process & DevOps" course?
Answer: Traditional DevOps pipelines focus solely on CI/CD code builds, while monitoring tools focus solely on alarms. This project closes the loop between Site Reliability Engineering (operations) and Agile sprint backlog execution (development) by automatically translating operational failures into prioritized Agile User Stories with explicit Definition of Done acceptance criteria.

Q4: How does the ticket close automatically without a scrum master manually changing its status?
Answer: Through Git Smart Commit parsing: when the developer implements the fix and includes `Fixes #1` in the commit message, GitHub Actions triggers the automated regression test suite. Upon 100% test pass, GitHub automatically links the commit SHA to Issue #1 and transitions its state from Open to Closed (Done).